

ESP32 Accident Detection & Alert System

An academic IoT prototype using **ESP32 + MPU6050 + NEO-6M GPS + Telegram Bot** to detect a possible high-impact event and send an emergency notification with the latest available GPS coordinates.

Project Status: Reconstructed prototype implementation. Test it on the actual hardware before claiming measured results or presenting it as the original firmware.

1. Features

- ESP32-based embedded controller
- MPU6050 acceleration monitoring
- Configurable accident-detection threshold
- NEO-6M GPS location acquisition
- Telegram emergency notification
- Google Maps link when GPS coordinates are valid
- Optional buzzer alert
- Serial Monitor debugging

2. Hardware Components

Component	Purpose
ESP32	Main controller and Wi-Fi connectivity
MPU6050	Acceleration / IMU sensing
NEO-6M GPS	Location acquisition
Buzzer	Local alert indication
Breadboard / wires	Prototyping and connections
USB / power source	Power and programming

3. System Flow

```
MPU6050 -> ESP32 -> Threshold Detection -> Buzzer
|
+--> GPS -> Wi-Fi -> Telegram
```

4. Pin Connections

MPU6050 → ESP32

MPU6050	ESP32
VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22

NEO-6M GPS → ESP32

NEO-6M	ESP32
TX	GPIO 16 (RX2)
RX	GPIO 17 (TX2)

GND	GND
VCC	Use voltage appropriate for the exact module

Buzzer: Signal/+ → GPIO 25; GND/- → GND.

Verify all connections against your actual hardware before powering the circuit.

5. Software & Libraries

- Arduino IDE
- ESP32 board support
- C/C++ Arduino framework
- MPU6050 library
- TinyGPSPlus library
- UniversalTelegramBot library

6. Installation

1. Install Arduino IDE and ESP32 board support.
2. Install MPU6050, TinyGPSPlus and UniversalTelegramBot through Library Manager.
3. Open `src/accident_detection.ino`.
4. Replace the Wi-Fi and Telegram placeholders locally.
5. Select the correct ESP32 board and COM/USB port.
6. Upload the sketch and open Serial Monitor at 115200 baud.

7. Telegram Setup

1. Create a Telegram bot through the official BotFather account.
2. Obtain the bot token and destination chat ID.
3. Put them into the local sketch.
4. Test Telegram messaging before testing the accident event.

Never publish the bot token, Wi-Fi password, or other credentials.

8. Working Principle

1. ESP32 initializes the MPU6050, GPS, buzzer and Wi-Fi.
2. MPU6050 acceleration on X/Y/Z is read continuously.
3. The firmware calculates acceleration magnitude.
4. If it crosses `ACCIDENT_THRESHOLD_G`, a possible accident is detected.
5. The buzzer activates.
6. GPS data is collected for a short period.
7. Telegram receives an emergency message and, if available, a Google Maps link.

9. Calibration

The example starts with:

```
const float ACCIDENT_THRESHOLD_G = 2.5;
```

This is **not a validated real-world accident threshold**. Calibrate and test it with the actual hardware. A robust safety system should consider multiple sensor features and event context rather than one threshold.

10. Testing Checklist

Test	Expected Result
Power on	ESP32 starts
Wi-Fi available	ESP32 connects
MPU6050 connected	Connection reported
GPS outdoors	Coordinates eventually available
Normal movement	No alert
Threshold event	Buzzer activates
GPS fix + event	Telegram contains location
No GPS fix	Telegram reports unavailable location

11. Limitations

- GPS can take time to acquire a fix.
- Single-threshold detection can create false positives/negatives.
- Telegram requires internet connectivity.
- The prototype uses `client.setInsecure()`; production systems should use proper TLS certificate validation.
- This is not a certified automotive or emergency-response device.

12. Future Scope

- Sensor fusion using acceleration + gyroscope
- Manual SOS/cancel button
- GSM/LTE connectivity
- Cloud logging
- Mobile dashboard
- Multi-stage accident classification
- Dataset-based ML classification
- False-alert cancellation window

13. Skills Demonstrated

Embedded C/C++, ESP32, sensor interfacing, I2C, UART/GPS, Wi-Fi, Telegram Bot API, IoT architecture, debugging and testing.

14. Repository Structure

```
ESP32-Accident-Detection-Alert-System/  
■■■■ README.md  
■■■■ src/  
■ ■■■■ accident_detection.ino  
■■■■ circuit/  
■ ■■■■ PIN_CONNECTIONS.md  
■ ■■■■ ADD_YOUR_CIRCUIT_DIAGRAM_HERE.txt  
■■■■ images/  
■ ■■■■ ADD_YOUR_PROJECT_PHOTOS_HERE.txt  
■■■■ docs/  
■ ■■■■ ADD_YOUR_REPORT_HERE.txt  
■■■■ .gitignore  
■■■■ LICENSE
```

15. Author

Anjali Kumari

B.Tech – Electronics & Communication Engineering
Galgotias College of Engineering and Technology

16. Disclaimer

Academic/educational prototype only. It is not a certified automotive, medical, or emergency-response device.